Branch Sums

AlgoExpert · Easy · Binary Trees

The problem

- List the root-to-leaf path sums, ordered left to right.

Intuition

- Carry the running sum down each path; at a leaf, that sum is one branch sum.

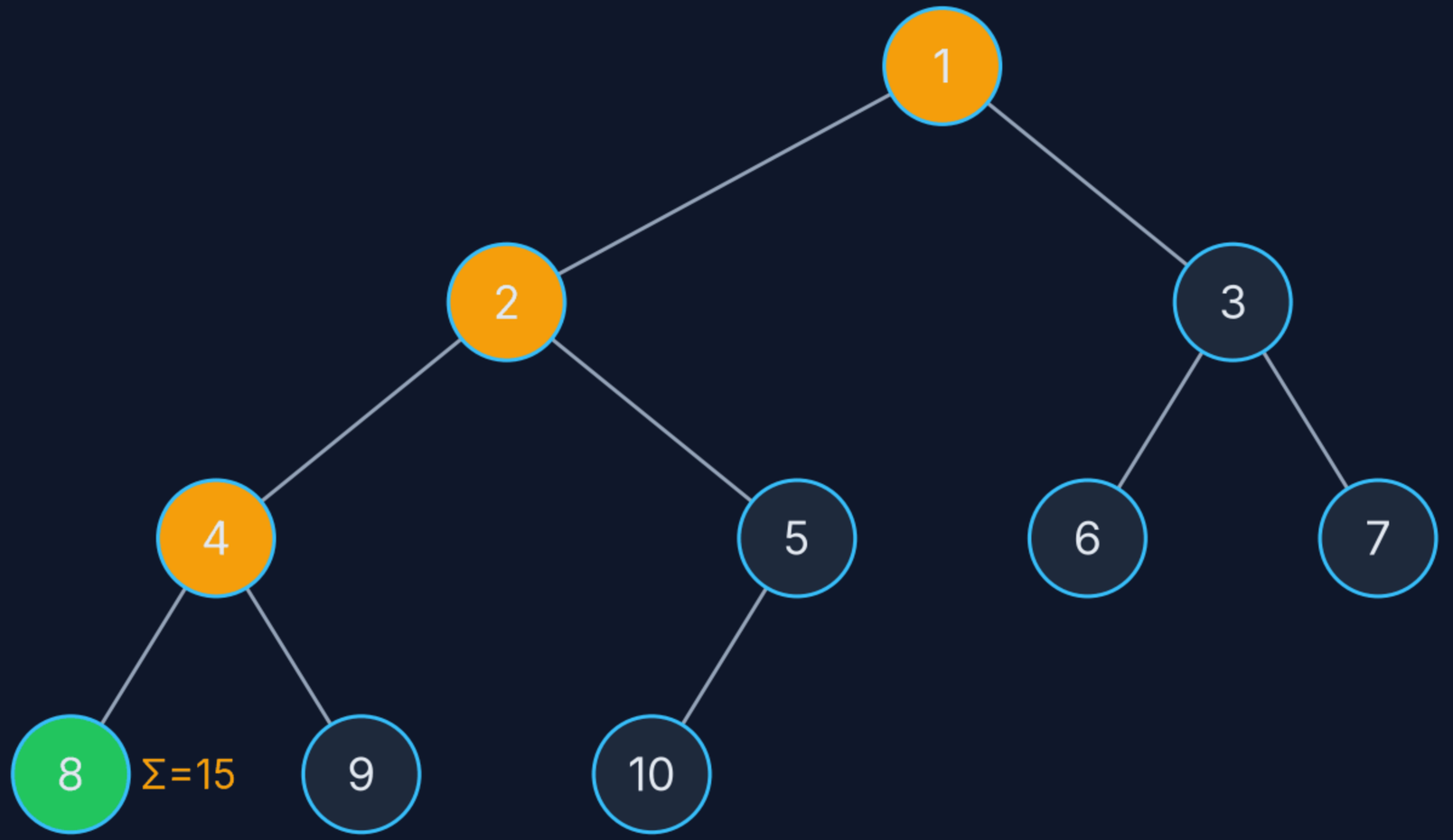
Brute force vs optimal

- Naive: collect each root-to-leaf path, then sum it → O(n·h)
- Optimal: carry the running sum down → O(n)
- Pre-order DFS keeps left-to-right order

Approach

- 1. Depth-first from the root with sum zero
- 2. Add the node's value
- 3. At a leaf, record the sum
- 4. Recurse left then right to keep left-to-right order

Worked example

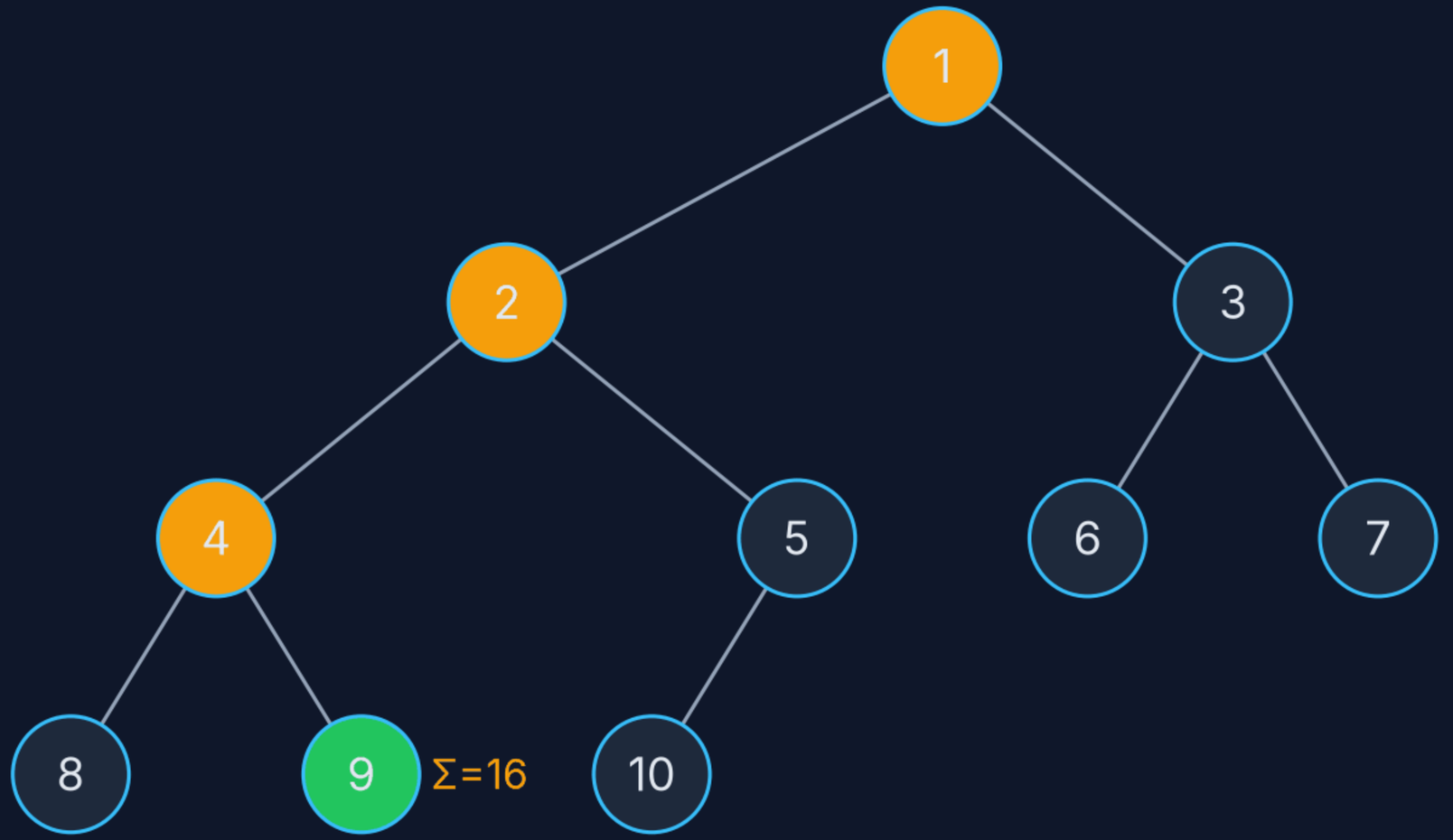



```
sums:

15
```

Leaf 8: branch sum 1 + 2 + 4 + 8 = 15

Worked example



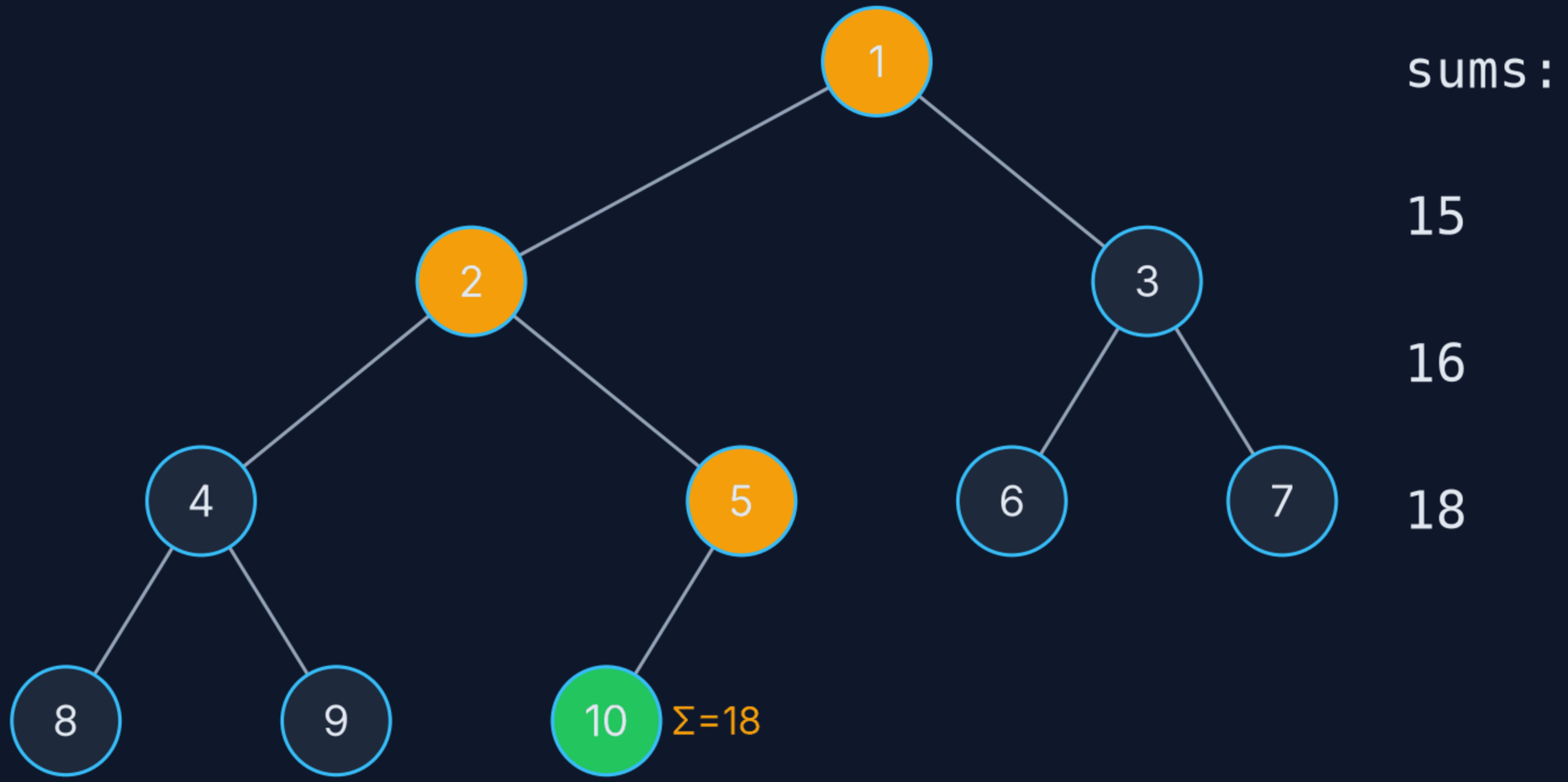
sums:

15

16

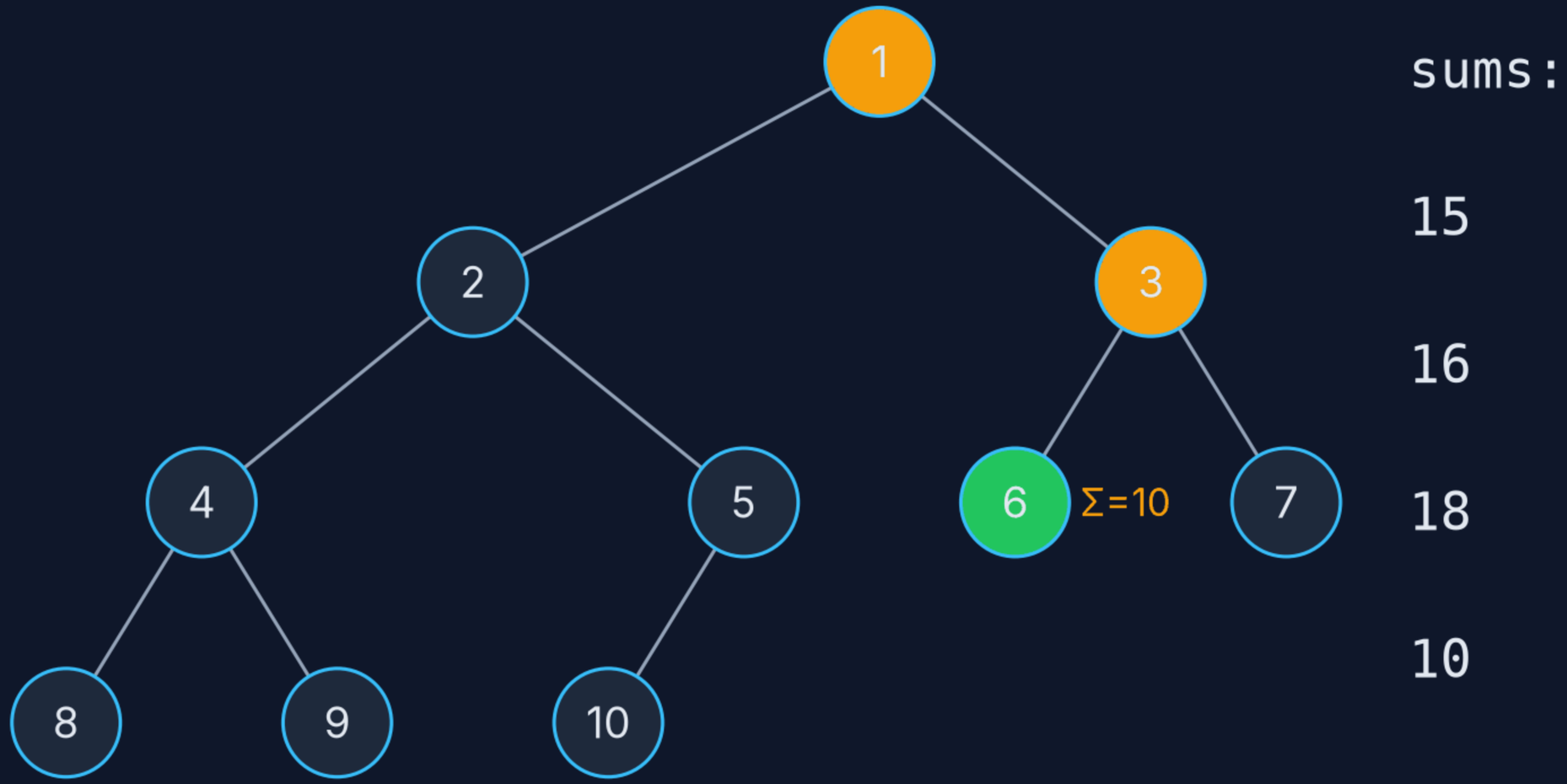
Leaf 9: branch sum 1 + 2 + 4 + 9 = 16

Worked example



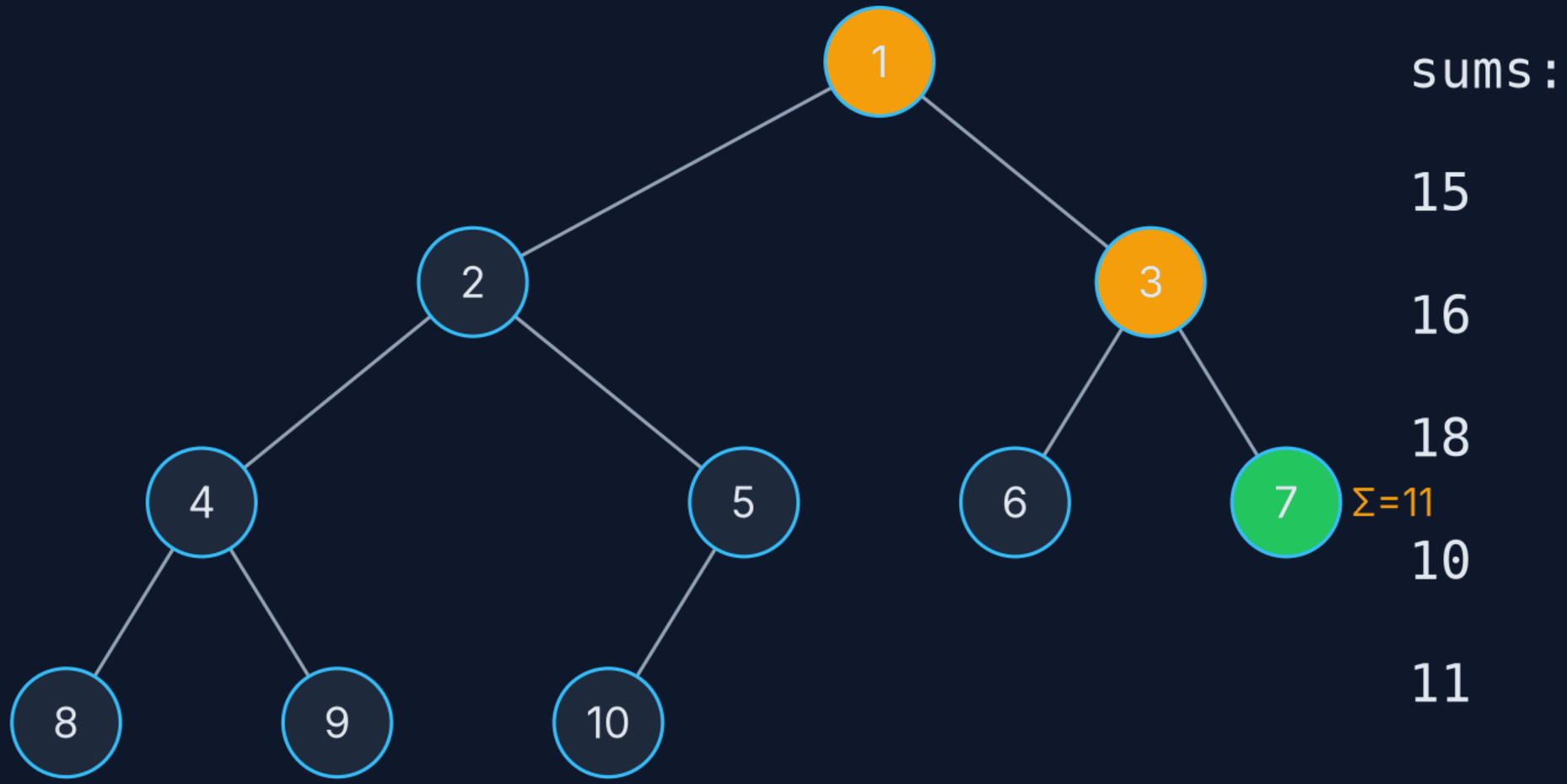
Leaf 10: branch sum 1 + 2 + 5 + 10 = 18

Worked example



Leaf 6: branch sum 1 + 3 + 6 = 10

Worked example



Leaf 7: branch sum 1 + 3 + 7 = 11

Java solution

```java
public class BranchSums {
    public static List<Integer> branchSums(BinaryTree root) {
        List<Integer> sums = new ArrayList<>();
        dfs(root, 0, sums);
        return sums;
    }

    private static void dfs(BinaryTree node, int running, List<Integer> sums) {
        if (node == null) return;
        int s = running + node.value;
        if (node.left == null && node.right == null) { sums.add(s); return; }
        dfs(node.left, s, sums);
        dfs(node.right, s, sums);
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- A node is a leaf only if BOTH children are null
- Single-node tree → [root.value]
- Recursion depth = tree height

Interview tips & follow-ups

- Follow-up: return the paths themselves
- Follow-up: path sum equals target?
- Iterative version with an explicit stack

Recap

- Carry the running sum down each path; at a leaf, that sum is one branch sum.
- Complexity: O(n)